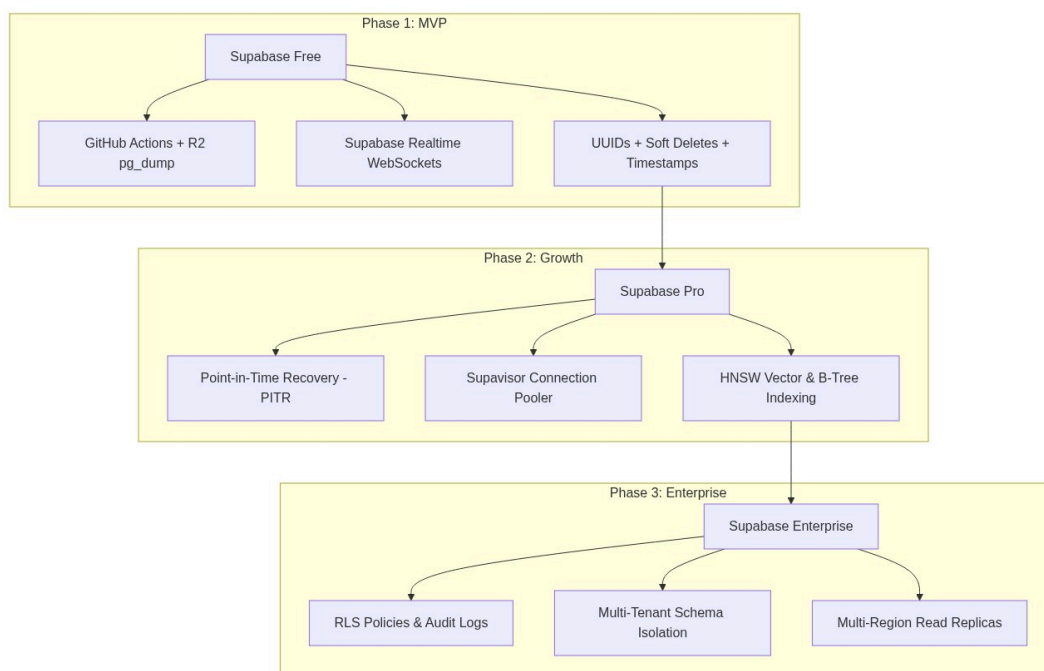


HireSense: Database Phase-Based Scaling & Architectural Roadmap

This document outlines the architectural blueprint and operational roadmap for scaling the HireSense Supabase (PostgreSQL) database. It is designed to guide your development from the early prototype phase to a high-performance, enterprise-grade B2B SaaS platform without introducing breaking database or workflow changes.

Architectural Phases at a Glance



Phase 1: MVP & Early Traction (0 - 1,000 Users)

Focus: Velocity, cost-efficiency (\$0/mo), and robust foundation building.

At this stage, your app runs on the Supabase Free Tier. You do not have built-in automated backups, but you enforce strict coding patterns that make future scaling trivial.

1. Database Architecture & Design System

Before deploying, every table schema in your database must adhere to the following template. This ensures data is clean, traceable, and easily mergeable in the future.

```
-- 1. Enable the UUID extension (native in Supabase)
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
```

```
-- 2. Master Table Template
CREATE TABLE student_profiles (
    -- Globally unique ID prevents primary key collisions during backup restores
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    -- Business data columns
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    resume_url TEXT,
    skills TEXT[],
    match_score INT DEFAULT 0,

    -- Audit & State Tracking Columns
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL,
    deleted_at TIMESTAMP WITH TIME ZONE DEFAULT NULL
);
```

The Automated `updated\_at` Trigger

Do not rely on your FastAPI backend code to update the `updated_at` timestamp. Let PostgreSQL handle it automatically:

```
-- Create the trigger function
CREATE OR REPLACE FUNCTION update_modified_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ language 'plpgsql';

-- Attach the trigger to your table
CREATE TRIGGER update_student_profiles_changestamp
    BEFORE UPDATE ON student_profiles
    FOR EACH ROW
    EXECUTE FUNCTION update_modified_column();
```

2. Soft-Delete Architectural Pattern

Instead of physically deleting candidate application records (which makes debugging and historic analysis impossible), your FastAPI application will execute logical deletes.

Read Query in FastAPI (database-level soft-delete filtering):

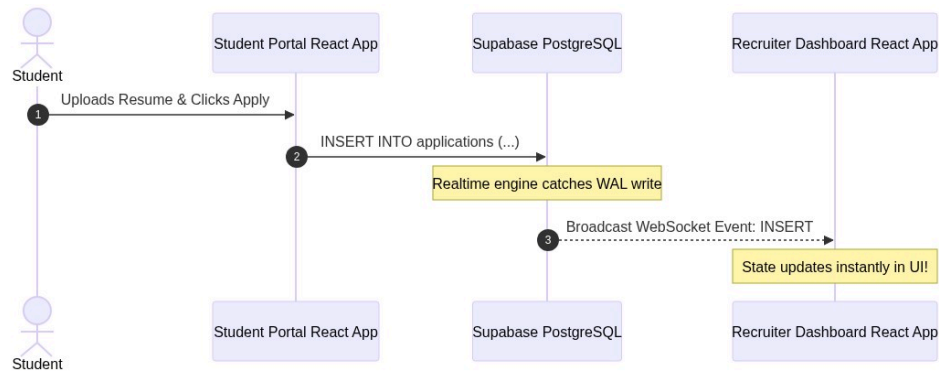
```
# python backend select query example
def get_active_students(db_client):
    response = db_client.table("student_profiles") \
        .select("*") \
        .is_("deleted_at", "null") \
        .execute()
    return response.data
```

3. Backup Strategy

- Pipeline: GitHub Actions scheduled cron runner (0 2 * * *) + pg_dump + Cloudflare R2 S3 storage API.
- Cost: \$0 (Within free tiers of GitHub and Cloudflare 10GB).
- Retention Policy: 30-day rolling deletion rule configured inside Cloudflare R2.
- Restore Runbook: free_backup_pipeline_guide.md

4. Real-time Synchronization Architecture

For the upcoming Student Portal and Recruiter Application Dashboard, you will use Supabase Realtime WebSockets to auto-update the recruiter's screen whenever a new student profile is parsed.



Phase 2: Growth & Commercialization (1,000 - 10,000 Users)

Focus: Performance scaling, high-speed matching pipelines, and professional Disaster Recovery.

At this stage, you upgrade to Supabase Pro (\$25/month). The connection string remains identical, but the infrastructure shifts underneath to support your growth.

1. Upgrade to Point-in-Time Recovery (PITR)

- The Upgrade: Switch from daily custom GitHub dumps to Supabase's native PITR backups.
- Mechanism: PostgreSQL physical write-ahead log (WAL) archiving.
- Benefit: You can restore your database to any exact second in the last 7 days.
- Downtime Protection: Eliminates "data loss window" entirely.

Daily Dump Architecture (Phase 1):

[Backup at 2:00 AM] -----> [Crash at 10:00 PM] = 20 hours data lost!

Point-in-Time Recovery (Phase 2):

[WAL Event] -> [WAL Event] -> [Crash at 10:00:15 PM]

Restore database to: 10:00:14 PM = ZERO data lost!

2. High-Performance Connection Pooling (Supavisor)

When hundreds of students and recruiters connect simultaneously, your database can run out of direct connections.

- Transition: In your database.py, update the connection port from 5432 (Direct PostgreSQL connection) to 6543 (Supavisor Transaction Pooler).

- Why: It allows thousands of concurrent API requests to share a small pool of physical database connections, preventing "Too many connections" backend errors.

3. Vector Database & B-Tree Indexing

As candidate search grows, query speeds will drop if you do not index.

- Text/B-Tree Indexes: Optimize standard lookup filters.
- Vector (HNSW) Indexes: Speed up AI resume matching.

```
-- Speed up email lookups & status queries
CREATE INDEX idx_student_email ON student_profiles(email);
CREATE INDEX idx_applications_status ON applications(status) WHERE deleted_at IS NULL;

-- Enable Vector matching (pgvector)
CREATE EXTENSION IF NOT EXISTS vector;

-- Speed up cosine similarity searches on 1536-dimension AI resume embeddings (OpenAI standard)
CREATE INDEX idx_student_embeddings_hnsw ON student_profiles
USING hnsw (resume_embedding vector_cosine_ops);
```

Phase 3: High Scale & Enterprise (10,000+ Users)

Focus: Enterprise security compliance, multi-tenancy, and low-latency global replication.

1. Row-Level Security (RLS) Policies

In a multi-tenant B2B SaaS, a recruiter from *Company A* must never see candidates from *Company B*. PostgreSQL Row Level Security enforces this directly at the database engine level, acting as a final shield even if there is a bug in your FastAPI backend.

```
-- 1. Enable RLS on the table
ALTER TABLE applications ENABLE ROW LEVEL SECURITY;

-- 2. Create a Policy allowing only authorized users to view applications
CREATE POLICY recruiter_isolation_policy ON applications
FOR SELECT
TO authenticated
USING (
    recruiter_id = auth.uid()
OR
    company_id = (SELECT company_id FROM profiles WHERE id = auth.uid())
);
```

2. Multi-Tenant Data Architecture Options

When scaling to large enterprises, decide between two primary isolation patterns:

Metric	Pattern A: Logical Isolation (Shared Schema)	Pattern B: Schema Isolation (Separate Database/Schema)
Isolation	Low	High
Scalability	Low	High
Complexity	Low	High
Cost	Low	High

| :--- | :--- | :--- |

| Structure | Single shared applications table with company_id columns. | Each client has a dedicated schema company_a.applications. |

| Cost | Extremely cheap (Standard Supabase plan). | Higher (Requires database replication management). |

| Security | Relies on RLS policies. | Physically separated at the database engine level. |

| Scaling | Easy to run global queries and analytics. | Complex to run cross-tenant aggregate analytics. |

- Recommendation for HireSense: Start with Logical Isolation (Shared Schema) combined with robust PostgreSQL RLS Policies. It is much cheaper, easier to maintain, and scales perfectly up to tens of thousands of active users.

3. Read Replicas for Analytics Isolation

Heavy screening reports or AI matching calculations can slow down active recruiters using the platform.

- The Architecture: Set up a Read Replica (a secondary database that mirrors your primary database in real-time).
- Traffic Routing:
- FastAPI Write requests (POST, PUT, DELETE) Primary Database.
- AI Matching Engines & Reporting Queries Read Replica Database.
- Result: Zero lag on the recruiter dashboard during heavy background AI parsing.

Implementation Strategy: Action Checklist

- [] Step 1: Run the updated_at trigger SQL on all current tables in your Supabase SQL Editor.
- [] Step 2: Change all tables to use UUID primary keys (id) instead of sequential integers.
- [] Step 3: Incorporate the .is_("deleted_at", "null") filter on all select statements in the FastAPI backend to implement soft-deletion.
- [] Step 4: Deploy the automated free daily backup workflow using Cloudflare R2 and GitHub Actions.
- [] Step 5: Enable Supabase Realtime toggle on the applications table to power your upcoming Student Portal UI updates.